Unit 7: Heaps (Week 6)

CPTR 242 — Sequential and Parallel Data Structures and Algorithms

Walla Walla University

🤔 You need the highest-priority item from a collection.

A sorted array gives O(1) access to the max, but O(n) insertion.

An unsorted array gives O(1) insertion, but O(n) to find the max.

Is there a structure that gives faster insert *and* access to extreme values?

7.1 The Heap

A **heap** is a complete binary tree satisfying the **heap property**.

- **Max-heap:** every node's value ≥ its children's values
- **Min-heap:** every node's value ≤ its children's values

```
Max-heap:         90
                 /  \
               55    70
              / \   / \
             10 20 40  30
```

The root always holds the **extreme value** — access is O(1).

Insert and remove cost **O(log n)** — proportional to tree height.

7.1 The Complete Tree Requirement

A binary tree is **complete** if every level is fully filled except possibly the last, which is filled **left to right**.

```
Valid (complete):      Invalid (not complete):
      A                        A
    /   \                    /   \
   B     C                  B     C
  / \   /                        / \
 D   E F                         E   F
```

This shape rule is not cosmetic. It's what makes O(log n) height guaranteed and array storage efficient.

> A complete binary tree with n nodes always has height $\lfloor \log_2 n \rfloor$.

🤔 A heap is a tree, but do we need pointers to store it?

The complete tree shape means there are no gaps in the structure.

If you laid out a complete binary tree level-by-level into an array, could you navigate parent–child relationships using only index arithmetic?

7.2 Array Representation

A heap maps perfectly onto an array. Using **1-based indexing** (root at index 1):

Relationship	Index
Left child of `i`	`2i`
Right child of `i`	`2i + 1`
Parent of `i`	`i // 2`

```
Array:  [ _ , 90, 55, 70, 10, 20, 40, 30 ]
Index:    0    1   2   3   4   5   6   7
```

No pointers needed!

7.2 Array Representation

In 0-based indexing (what Python lists and most real implementations use), index 0 is the root, and the formulas shift:

Relationship	Index
Left child of `i`	`2i + 1`
Right child of `i`	`2i + 2`
Parent of `i`	`(i - 1) // 2`

7.2 Sift-Up and Sift-Down

Two operations restore heap order after insertions and removals.

Sift-up — used after insert:
Compare node with its parent; swap upward while the heap property is violated.

Sift-down — used after remove:
Compare node with its largest child; swap downward while the heap property is violated.

Each swap moves one level in the tree. Height = log n → both operations cost **O(log n)**.

7.2 Sift-Up Example

Insert 95 into this max-heap:

```
Before:                After step 1:         After step 2:
     90                     90                    95
    /  \                   /  \                  /  \
  55    70       →       95    70       →      90    70
 /                      /                      /
95 ← 95 appended     55                     55
```

95 > 55: swap. 95 > 90: swap. 95 reaches the root. **2 swaps, O(log n).**

Part 3: Heap Operations in C++

Section 7.3

🤔 `extractMax` removes the root. That leaves a hole at index 1.

You can't just delete it and leave a gap, so the array must stay contiguous.

Which element should fill the root, and why that one specifically?

7.3 Insert and Sift-Up

```
void insert(int val) {
    data.push_back(val);
    siftUp(data.size() - 1);
}
```

Append the new value as the last element (maintaining complete-tree shape), then sift it up to its correct position.

7.3 extractMax and Sift-Down

```
int extractMax() {
    int top = data[1];
    data[1] = data.back();
    data.pop_back();
    siftDown(1);
    return top;
}
```

Move the **last element** to the root. This keeps the array contiguous and preserves the complete-tree shape. Then sift it down to restore heap order.

> Moving any other element would leave a hole mid-array. The last element is the only safe choice — it can be removed cleanly.

7.3 siftDown

```
void siftDown(int i) {
    int n = data.size() - 1, largest = i;
    if (2*i <= n && data[2*i] > data[largest])   largest = 2*i;
    if (2*i+1 <= n && data[2*i+1] > data[largest]) largest = 2*i+1;
    if (largest != i) { swap(data[i], data[largest]); siftDown(largest); }
}
```

Find the largest among the node and its two children. If the node isn't already largest, swap and recurse. Stops when heap order is restored or a leaf is reached.

Part 4: The Priority Queue ADT

Section 7.4

🤔 **An operating system schedules processes by urgency, not arrival order.**

A regular queue serves the item that waited longest.

What ADT serves the item with the highest priority, regardless of when it arrived?

7.4 Priority Queue ADT

A **priority queue** always provides access to the highest-priority item.

Operation	Description	Heap cost
`enqueue(val)`	Insert an item	O(log n)
`dequeue()`	Remove highest priority	O(log n)
`peek()`	View highest priority	O(1)
`isEmpty()`	Check if empty	O(1)

The heap is the canonical **implementation** of this abstraction because it matches every cost target exactly.

7.4 STL: `std::priority_queue`

```
#include <queue>
priority_queue<int> pq;          // max-heap by default
pq.push(30); pq.push(10); pq.push(50);
cout << pq.top();                // 50
```

For a **min-heap**:

```
priority_queue<int, vector<int>, greater<int>> pq;
```

Part 5: Heapsort

Section 7.5

🤔 You can extract the maximum from a heap in O(log n).

If you do that n times and collect the results, you get a sorted sequence.

What is the total time? And can you do the whole thing without any extra memory?

7.5 Heapsort: Two Phases

Phase 1 — Build the heap:

Turn an unsorted array into a max-heap in-place.

Phase 2 — Sort:

Repeatedly swap the root (max) with the last element, shrink the heap boundary by one, and sift-down.

```
After build:  [ 9  6  8  5  3  2  7  4  1 ]
After swap 1: [ 8  6  7  5  3  2  1  4 | 9 ]
After swap 2: [ 7  6  4  5  3  2  1 | 8  9 ]
...
Sorted:       [ 1  2  3  4  5  6  7  8  9 ]
```

The sorted portion (after `|`) grows right. No extra array needed — **O(1) space**.

7.5 Heapsort Complexity

Phase	Cost
Build heap (Floyd's)	O(n)
n extractions × O(log n) each	O(n log n)
Total	**O(n log n)**
Extra space	**O(1)**

Heapsort matches merge sort's O(n log n) guarantee while using O(1) space — unlike merge sort's O(n).

> Heapsort is not stable (equal elements may reorder). For stable O(n log n) sorting, use merge sort.

Part 6: Heapsort in C++

Section 7.6

🤔 Heapsort is O(n log n) worst-case with O(1) space.

Quicksort averages faster in practice despite the same asymptotic complexity.

What property of memory access could explain why quicksort beats heapsort on real hardware?

7.6 Why Quicksort Wins in Practice

Heapsort's sift-down jumps between a parent and a child far away in the array:

```
Parent at index 1 → children at index 2, 3
Parent at index 2 → children at index 4, 5
Parent at index 500 → children at index 1000, 1001
```

These are large memory strides, so the CPU cache cannot predict them. Each sift-down step is likely a **cache miss**.

Quicksort partitions **contiguous subarrays** — memory access is sequential and cache-friendly. In practice, quicksort runs 2–5× faster than heapsort despite identical O(n log n) complexity.

> Heapsort is preferred in embedded or real-time systems where quicksort's $O(n^2)$ worst case is unacceptable and memory is constrained.

Part 7: Treaps

Section 7.7

🤔 A BST's height depends entirely on insertion order.

Insert 1, 2, 3, 4, 5 in order and you get a linked list — O(n) height.

How could you prevent adversarial inputs from degrading a BST to O(n)?

7.7 Treap = Tree + Heap

A **treap** gives each node two values:

- A **key** — must satisfy BST order (left < node < right)
- A **random priority** — must satisfy max-heap order (node > children)

```
          (key=5, pri=90)
         /                \
 (key=3, pri=40)   (key=8, pri=70)
 /
(key=1, pri=20)
```

Both properties hold simultaneously. The random priorities determine the tree's shape — and since they're random, no input sequence of keys can force a bad shape.

7.7 Expected Height: O(log n)

Each (key, priority) pair has exactly one valid treap shape. The node with the highest priority must be the root — and since priorities are assigned randomly and independently of keys, the root is equally likely to be any of the n nodes.

This is equivalent to building a BST from a **random permutation** of keys, which has expected height **O(log n)** — regardless of the order keys actually arrive.

> An adversary who controls key insertion order cannot predict or influence priorities. The randomness belongs to the data structure, not the input.

7.7 Treap Insert

1. Insert the new node by **BST key** at the correct leaf position
2. Assign a **random priority**
3. **Rotate up** while the node's priority exceeds its parent's

```
Node* rotateRight(Node* y) {
    Node* x = y->left;
    y->left = x->right; x->right = y;
    return x;
}
```

Rotations preserve BST key order while fixing heap priority order. Expected O(log n) rotations.

7.7 Treaps vs. Balanced BSTs

Property	AVL / Red-Black	Treap
Worst-case height	O(log n) guaranteed	O(log n) expected
Insert complexity	O(log n) + rebalancing cases	O(log n) + rotations
Implementation complexity	High (many cases)	Low (rotations only)
Deterministic?	Yes	No (randomized)

Treaps trade the guarantee for simplicity. In practice they perform comparably to AVL trees with significantly less bookkeeping. Used in competitive programming and some production databases.

Lab 6 Wednesday! 😄 Have a great day!